1. Block Connectivity & Signal Routing Diagram

The audio processing thread moves array data arrays sequentially from block to block. The pointer changes between steps show how data transfers through shared static memory arrays:

Source Block	Output Array / Variable	Destination Block	Input Parameter Checked
I2S Microphone Driver	`input_block`	Hardware Buffer Manager	`pfInput`
Hardware Buffer Manager	`rb_out`	Parallel Biquads (LP, BP, HP)	`pfInput`
Biquad Array Combiner	`biquad_sum`	3-Band Equalizer (SW1 Control)	`pfInput`
3-Band Equalizer Splitting	`eq_low`, `eq_mid`, `eq_high`	Dynamic Band Gain Adjustments	`pfInput` / `pfOutput` (In-Place)
EQ Channel Mixer	`final_out`	Normal Linear Gain Stage (SW2 Control)	`pfInput`
Linear Gain Node	`gain_out`	Pure Time-Shift Delay (SW3 Control)	`pfInput`
Pure Time-Shift Delay	`delay_out`	Signal Amplitude Limiter	`pfInput`
Signal Amplitude Limiter	`pfOutput` (Shared)	I2S Amplifier Output Driver	`pfInput`

2. Loop Flows & Processing Code Snippets

Below are the specific implementation loops for each feature block. Each snippet uses print-safe layouts to maintain clean formatting on physical print or PDF exports.

A. Parallel Biquad Combination Filter Loop

Feature: Multi-Band Audio Splitting

Implementation: Audio samples from the hardware buffer feed into three separate biquad calculations simultaneously. The individual bands merge into a single stream using a scale factor loop to maintain steady system levels.

```
// Parallel execution across distinct filtering equations
low_pass_process(&bq_lp, rb_out, lp_out, AUDIO_BLOCK_SIZE);
band_pass_process(&bq_bp, rb_out, bp_out, AUDIO_BLOCK_SIZE);
high_pass_process(&bq_hp, rb_out, hp_out, AUDIO_BLOCK_SIZE);

// Linear combination normalization loop
for (uint32_t i = 0; i < AUDIO_BLOCK_SIZE; i++) {
    biquad_sum[i] = (lp_out[i] + bp_out[i] + hp_out[i]) * 0.3333f;
}
```

B. 3-Band Equalizer Matrix Routing (SW1)

Feature: Software Frequency Tuning

Implementation: When active, the stream routes through an equalizer topology block that uses in-place modifications to scale individual frequency ranges.

```
if (eq_enabled) {
    // 1. Split the unified sum into three baseline arrays
    equalizer_process(&eq_hdl, biquad_sum, eq_low, eq_mid, eq_high, AUDIO_BLOCK_SIZE);

    // 2. Adjust amplitude scales in-place inside individual arrays
    gain_process(&eq_gain_hdl.low,  eq_low,  eq_low,  AUDIO_BLOCK_SIZE);
    gain_process(&eq_gain_hdl.mid,  eq_mid,  eq_mid,  AUDIO_BLOCK_SIZE);
    gain_process(&eq_gain_hdl.high, eq_high, eq_high, AUDIO_BLOCK_SIZE);
```

```
    // 3. Recombine modified components back into the active path
    for (uint32_t i = 0; i < AUDIO_BLOCK_SIZE; i++) {
        final_out[i] = eq_low[i] + eq_mid[i] + eq_high[i];
    }
    dsp_out = final_out; // Update the active stream pointer
}
```

C. Normal Linear Gain Scaling Loop (SW2)

Feature: Normal Linear Gain

Implementation: Applies a straight linear multiplication scaling factor to each incoming sample without running any complex logarithmic or decibel conversions.

```
STATUS gain_process(
    gain_hdl_t *phdl,
    const float *pfInput,
    float *pfOutput,
    uint32_t ui32NumSamples)
{
    if (phdl == NULL || pfInput == NULL || pfOutput == NULL) return STATUS_NOT_OK;

    for (uint32_t i = 0; i < ui32NumSamples; i++) {
        // Normal linear amplification multiplication
        pfOutput[i] = pfInput[i] * phdl->gain_linear;
    }
    return STATUS_OK;
}
```

D. Pure Time-Shift Delay Loop (SW3)

Feature: Signal Time-Shifting

Implementation: Performs a clean, feed-forward, sample-by-sample time displacement. New data overwrites slots sequentially without any feedback accumulation or multi-reflection echoes.

```
STATUS delay_process(
    delay_hdl_t *phdl,
    const float *pfInput,
    float *pfOutput,
    uint32_t ui32NumSamples)
{
    if (phdl == NULL || pfOutput == NULL) return STATUS_NOT_OK;
```

```
    for (uint32_t i = 0; i < ui32NumSamples; i++) {
        float input_sample = (pfInput != NULL) ? pfInput[i] : 0.0f;

        // 1. Output the historic sample directly from the current slot
        pfOutput[i] = phdl->delay_line[phdl->write_idx];

        // 2. Overwrite the buffer with new data (No echo feedback loop)
        phdl->delay_line[phdl->write_idx] = input_sample;

        // 3. Step our pointer forward and wrap around at array boundaries
        phdl->write_idx++;
        if (phdl->write_idx >= phdl->delay_samples) {
            phdl->write_idx = 0;
        }
    }
    return STATUS_OK;
}
```

E. Audio Output Safety Limiter

Feature: Clipping & Speaker Protection

Implementation: A final hardware protection step that keeps signal amplitudes within set boundaries. It clamps the alternating positive and negative peaks of the audio waveform before it goes to the speaker amplifier.

```
STATUS limiter_process(
    limiter_hdl_t *phdl,
    const float *pfInput,
    float *pfOutput,
    uint32_t ui32NumSamples)
{
    if (phdl == NULL || pfInput == NULL || pfOutput == NULL) return STATUS_NOT_OK;

    for (uint32_t i = 0; i < ui32NumSamples; i++) {
        float sample = pfInput[i];

        // Clamp the upper and lower bounds of the alternating audio waveform
        if (sample > phdl->fThreshold) {
            sample = phdl->fThreshold;     // +thresh ceiling clamp
        } else if (sample < -phdl->fThreshold) {
            sample = -phdl->fThreshold;    // -thresh floor clamp
        }

        pfOutput[i] = sample;
    }
```

```
    return STATUS_OK;
}
```

3. Runtime Feature Configurations

This architecture combines static hardware allocation with flexible software processing switches. This balance meets key real-time constraints:

- **Dynamic Bypass Safety:** Disabling an effect (such as the Delay block) does not break code execution. The module continues to manage its internal circular buffer indices seamlessly while routing the clean dry audio path instantly to prevent audio dropouts.
- **Memory Isolation:** Shared computing structures are split into independent configuration vectors and runtime handles. This approach separates state tracking data from system code blocks, keeping the pipeline stable during long periods of operation.